OutOfRole in Plain Words

The Debugging Table · IBM Z Datathon 2026 · AI Secured: Innovation Without Exposure

In one paragraph

OutOfRole compares each employee's recent activity with their own normal weeks, and gives security analysts a short daily review list of 20 names. For each person on the list, it gathers the evidence that is new to that person and rare in the company. In the LinuxONE build, IBM Granite will pick a summary label and the facts that support it. Code writes every fact sentence and rejects any reply that breaks the rules. Separately, a Netflix-style recommender predicts which websites each person will use next, so routine access can be pre-approved, and websites people stopped using become suggestions for access review. Processing and AI stay on the one server, and people appear only as pseudonyms.

The problem

Some people misuse access they legitimately have. They copy files to a USB stick before quitting, upload documents to a leak site, or log into colleagues' computers to snoop. Valid logins can hide this misuse, and analysts need a short list they can actually review.

What OutOfRole does

1. **Watch.** Every day, compare each person's last 7 days with every one of their normal weeks from training: USB connections, file copies, after-hours activity, and the number of computers used. When an activity beats all their normal weeks, their score rises. The 20 highest scores form the day's review list.
2. **Explain.** For each name on the list, rank what they did: things not seen in their training history first, then the rarest in the company. Granite replies with a label from a fixed list and the IDs of the facts behind it. Code renders each fact from the data. Any other reply is rejected, and a plain fallback is shown instead.
3. **Prevent.** The recommender learns which websites each person uses and predicts the ones they will want next. Those can be pre-approved; anything else goes to a reviewer. Sites no one in the company visited during the training months are blocked. At the end of a review period, websites a person used in training but not since become access-review suggestions. CERT has no real permission lists, so past use stands in for held access.

What we measured

The data is the CERT insider-threat benchmark, in two releases. r4.2 has 1,000 staff and 70 insiders; r5.2 has 2,000 staff and 99 insiders. The method behind the daily list was chosen after exploratory testing. Build metrics come from one program, `outofrole_minimal.py`; the historical controls (chance, the method we named in advance) come from the experiment scripts.

Step	Result	r4.2	r5.2
Watch	Insiders on the daily list before their last malicious action	59 of 70	**80 of 99**
	Same, standard 28-day baseline, also 20 names a day	59 of 70	40 of 99
	Share of each insider's malicious actions on later days (all insiders, missed = 0)	54%	48%
	Normal staff ever placed on the list over the test period	230 of 930	291 of 1,901

Step	Result	r4.2	r5.2
Explain	Insiders caught in time whose top 3 evidence items include an activity matching the answer key	64%	85%
	Same, ranking by rarity alone	49%	51%
	Strong evidence at first alert, insiders vs normal staff (no labels used)	29% vs 18%	65% vs 31%
Prevent	Next new websites caught by the recommender's top 20 (popularity in brackets)	21% (14.5%)	19% (13%)
	Access requests per person per month: own history only, then with the recommender	2.31 to 1.82	3.15 to 2.55
	Websites used in training but not again (access-review suggestions)	6.0%	3.6%
	Wikileaks and keylogger-site visits blocked by the new-to-company rule	100%	100%

What this means:

- **On r5.2, the release we held back for testing, the daily list catches twice as many insiders as the standard baseline.** Both lists hold 20 names a day, though ours changes a little faster (11.3% vs 8.9% of names a day). On r4.2 the two tie.
- **The recommender is not the detector.** It predicts which websites people will use next better than popularity does, and that is the job it does well.

How simple it is

Spark turns the raw logs into three tables. Everything else is two short Python files:

- `outofrole_minimal.py` (214 lines, about 20 seconds per release on a laptop): the daily list, the evidence ranking, the recommender, the access suggestions and the block rule.
- `case_file.py` (91 lines): the evidence pack, the Granite prompt, the reply check and the plain fallback. Its self-test feeds in six bad replies (an invented number, an uncited claim, an outside address, an unknown fact, non-JSON text and a label outside the list); all six are rejected.

Granite and a small dashboard sit on top.

How it connects to our proposal

We promised	We deliver
Catch insiders with valid logins without flooding analysts	A 20-name daily list
"Learns what each employee normally uses"	Each person's own normal weeks; a recommender learns their websites
"Access it would never suggest is flagged"	Recommended access is pre-approved; anything else goes to review; never-seen sites are blocked
"An AI on the same server explains why"	Granite labels each case and cites the supporting facts; code renders them; all on LinuxONE (built at the event)
Analysts: a short, ranked list of explained alerts	The daily list plus case files
Compliance: a record of who accessed what and why	Each case file lists dated, cited evidence items
Access admins: permissions held but never used	Access-review suggestions from past use (6.0% and 3.6% of websites), plus pre-approval suggestions
Data never leaves the platform	One server, pseudonyms only
Spark matrix, ALS on early months, later months replayed as a live stream	Spark tables and ALS on the early months, as promised; the later months are replayed day by day

What changed: our tests showed that each person's own history catches insiders better than the recommender does. So behavioural history now raises the alerts, and the recommender moved to access advice, where it measurably helps. The computer recommender showed no benefit, and a file recommender was not built.

What we say openly

- CERT is synthetic data. These are replay results, not a live deployment.
- The method behind the daily list was chosen after seeing results. The method we had named in advance also beat the baseline (54 vs 28 of 99).
- Evidence is graded on the day each insider was caught in time. 46% (r4.2) and 31% (r5.2) of insiders were first listed before their labelled attack began, when there was no labelled malicious activity to cite yet.
- Two-day sysadmin sabotage is mostly missed.
- Granite and the LinuxONE run are built at the event. The code they plug into is tested here.

Every number above, with the script that produced it, is in `feasibility/Verified_Numbers.md`.